

Atrial Fibrillation Detection & Context-Aware Wearable False-Alarm Analysis

A Comprehensive, Beginner-Friendly Technical Guide to the Complete Machine Learning Pipeline

Project: B.Tech Final Year Capstone	Dataset: CACHET-CADB (Wearable Multi-sensor)
Core Task: 10-second ECG Arrhythmia Detection	Validation: 5-Fold Subject-Wise GroupKFold
Models: Support Vector Machine & XGBoost	Status: Fully Validated Pipeline (Phases 1-8 + Quality Gate)

1. Project Overview & Clinical Motivation

What is Atrial Fibrillation (AF)? In a healthy heart, electrical signals travel in an orderly fashion from the top chambers (atria) to the bottom pumping chambers (ventricles), causing rhythmic, synchronized heartbeats known as *Normal Sinus Rhythm (NSR)*. In Atrial Fibrillation, the electrical signals in the atria fire chaotically. Instead of beating cleanly, the top chambers quiver or fibrillate. This causes the ventricles to beat irregularly and often rapidly. Blood can pool in the quivering atria, form clots, travel to the brain, and cause an ischemic stroke. In fact, AF increases stroke risk by **five times**.

The Wearable Monitoring Revolution: Modern smartwatches and wearable patches (e.g., Apple Watch, Fitbit, single-lead ECG patches) allow continuous cardiac monitoring during everyday life. Instead of wearing a bulky 12-lead hospital holter for 24 hours, people can monitor their rhythm for days or weeks in natural environments.

The Wearable Crisis — False Alarms: Wearable monitors suffer from a major clinical challenge: **false alarms**. When an ambulatory patient walks, exercises, brushes their teeth, or bumps the sensor, mechanical movement and skin-electrode friction introduce *motion artifacts* (electrical noise). If a smart device misidentifies noise as atrial fibrillation, the patient receives an urgent notification. This triggers severe anxiety, unnecessary emergency room visits, expensive clinical workups, and eventually 'alert fatigue' where users turn off notifications entirely.

PRIMARY RESEARCH GOAL

Most published biomedical research papers only report overall model accuracy on clean laboratory recordings. This project is unique because it directly tackles the real-world failure mode: **What happens when wearable sensors encounter motion noise, and how can we design a smart quality gate to prevent false alarms before they reach the user?**

2. Dataset Structure & Phase 1 Indexing

The project uses the publicly available **CACHET-CADB** (Copenhagen Actigraph and Cardiac Event Team Cardiac Arrhythmia Database). It captures continuous single-lead ECG recordings alongside physical activity context (3-axis acceleration, step counts, and physical posture).

File Formats in the Dataset: Each patient folder contains data in the open Unisens format: • `unisens.xml`: An XML metadata descriptor declaring channels, data types, and sampling frequencies. • `ecg.bin`: Raw 16-bit signed integer

binary files storing the recorded electrical voltages. • `acc.bin`: 3-axis accelerometer readings at 64 Hz. • `context.xlsx`: Smartphone diary logs of activities and posture. • `annotation.csv`: Ground-truth expert clinical labels assigning timestamps and diagnostic codes to specific 10-second segments.

Hard-Won Empirical Invariants (Ground Truths)

Before building machine learning algorithms, careful inspection of the raw data revealed several critical facts that directly contradicted initial textbook assumptions. Documenting and verifying these facts was essential to prevent silent data corruption:

Parameter	Correct Finding	Why It Matters & Evidence
ECG Sample Rate	1024 Hz (downsampled to 256 Hz)	Every <code>unisens.xml</code> declares <code>sampleRate='1024'</code> . An early config assumed 256 Hz, which would have distorted all heart-rate and time-interval calculations by exactly 4x.
Class Label Mapping	Class 1 = AF Class 2 = NSR	Intuition assumes 1 means normal. The opposite is true: Class 1 segments have RMSSD 290 ms (irregular AF) while Class 2 has 42 ms. Furthermore, healthy control subjects (PNSR-1, 3, 4) have 0 Class 1 segments and 65 Class 2 segments.
Segment Length	10 Seconds (10,240 samples @ 1024 Hz)	Annotations strictly correspond to 10-second windows. Merging segments into 5-minute blocks would mix conflicting labels. 10s is too short for frequency-domain HRV (LF/HF), so frequency features are disabled.
Ectopic Beat Filter	Disabled (False)	Standard HRV filters remove beats deviating >20% from neighbor beats. In AF, beats are naturally irregular. Enabling the filter deleted 48.5% of AF beats and destroyed 25.7% of AF segments!
Cohort Demographics	23 Usable Subjects (P1-P19, P21, P23, PNSR-1, 3, 4)	P20 and P22 do not exist; P16 lacks annotations. PNSR-* are healthy controls. Testing must use subject-wise splits to prevent data leakage.

Phase 1 Implementation: The script `src/preprocessing/dataset_index.py` executes `build_segment_index()`. It recursively indexes the filesystem, validates start/stop sample indices against the file lengths, extracts patient IDs, and outputs `data/processed/segment_index.csv` containing **1,602 annotated segments** (747 AF, 615 NSR, 221 Noise, 19 Other).

3. Signal Preprocessing (Phase 2)

Raw ECG signals captured by wearable sensors are contaminated by multiple environmental and biological noise sources: baseline wander (breathing causing the baseline to drift up and down), powerline interference (50 Hz electromagnetic hum from wall outlets), and electromyographic (EMG) noise (high-frequency spikes from muscles tensing). Phase 2 cleans the raw signal using `src/preprocessing/ecg_filter.py`.

Step 1 — Converting ADC integers to physical millivolts (mV): The raw binary `.bin` files store voltages as digitized 16-bit integers. Using `src/preprocessing/unisens_reader.py`, we read the sensor's calibration scale and baseline offset from `unisens.xml` to convert raw integer counts into physical millivolts.

Step 2 — Butterworth Bandpass Filter (0.5 to 40 Hz): The function `bandpass_filter()` implements a 4th-order Butterworth digital filter. A bandpass filter acts like a selective audio equalizer: it allows frequencies between 0.5 Hz and 40 Hz to pass through while silencing everything else. The low cutoff of 0.5 Hz eliminates slow breathing wander. The high cutoff of 40 Hz removes muscle tremor noise. We apply it using `scipy.signal.filtfilt`, which runs the filter forwards and backwards (zero-phase filtering). This ensures that R-peaks do not shift even a fraction of a millisecond in time!

Step 3 — 50 Hz Notch Filter: The function `notch_filter()` applies a specialized infinite impulse response (IIR) notch filter at exactly 50 Hz ($f_Q=30^\circ$). It acts as an ultra-narrow 'acoustic trap' that silences the 50 Hz AC electrical grid hum without altering cardiac signals at 49 Hz or 51 Hz.

Step 4 — Downsampling to 256 Hz: The raw signal arrives at 1024 Hz (1,024 samples per second). The function `downsample_ecg()` uses `scipy.signal.resample` to downsample to 256 Hz. 256 samples per second provides sub-4-millisecond timing precision — well beyond what is needed to detect heartbeats — while reducing memory and computation requirements by **75%**.

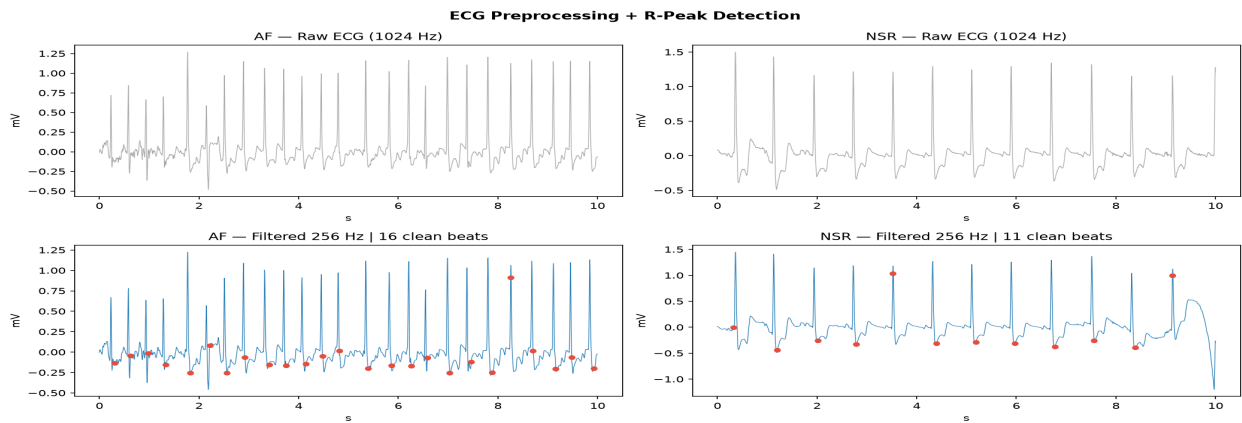


Figure 1: Raw ECG waveform vs cleaned 0.5–40 Hz bandpass and 50 Hz notch filtered signal at 256 Hz.

4. R-Peak Detection & HRV Feature Extraction (Phase 3)

What is an R-Peak? Each heartbeat generates a sequence of electrical waves on the ECG: the P wave (atria activating), the QRS complex (ventricles contracting to pump blood), and the T wave (ventricles relaxing). The **R-peak** is the tallest, sharpest upward spike of the QRS complex. The distance between two consecutive R-peaks is called the **RR interval** (or Normal-to-Normal / NN interval), measured in milliseconds. By measuring the variation in these time intervals, we compute **Heart Rate Variability (HRV)**.

The Pan-Tompkins Algorithm (`src/peak_detection/rpeak_detector.py`): To find R-peaks automatically, we implemented the industry-standard Pan-Tompkins algorithm. It processes the ECG in five mathematical stages:

- Bandpass Filtering:** Isolates frequencies between 5 and 15 Hz, maximizing the energy of the QRS complex while rejecting P and T waves.
- Differentiation:** Computes the derivative ($\frac{dy}{dt}$), highlighting the steep rising edge of the R-peak.
- Squaring:** Squares every value (y^2), making all values positive and exponentially boosting large QRS peaks over small noise fluctuations.
- Moving Window Integration:** Smooths the signal with a 150 ms rolling window, creating a smooth energy envelope.
- Adaptive Thresholding:** Dynamically tracks the signal peak amplitude and noise floor. If a peak crosses the threshold and occurs at least 200 ms after the previous peak (the physiological refractory period, since human hearts cannot beat >300 bpm), it is registered as an R-peak.

The 20 Extracted HRV Features (`src/feature_extraction/hrv_features.py`)

From each 10-second segment (which contains approximately 10 to 18 heartbeats), we compute 20 distinct numerical features across three mathematical domains:

Category	Feature Name	Clinical & Mathematical Meaning
----------	--------------	---------------------------------

Time Domain (Averages & Spread)	MeanNN MedianNN SDNN RMSSD SDSD IQRNN MadNN	• MeanNN / MedianNN: Average and median interval between beats (ms). • SDNN: Standard deviation of intervals (overall rhythm spread). • RMSSD: Root Mean Square of Successive Differences (short-term beat-to-beat bounce). • SDSD: Standard deviation of successive differences. • IQRNN / MadNN: Interquartile range and median absolute deviation (robust outlier-resistant spread metrics).
Time Domain (Relative & Thresholds)	CVNN CVSD MCVNN pNN50 pNN20	• CVNN: Coefficient of Variation ($\text{\\$SDNN} / \text{\\$MeanNN\\$}$) — normalizes spread by heart rate. • MCVNN: Median-based coefficient of variation ($\text{\\$IQRNN} / \text{\\$MedianNN\\$}$). • pNN50 / pNN20: Percentage of adjacent heartbeats that differ by more than 50 ms or 20 ms. In healthy resting sinus rhythm, pNN50 is low; in AF, adjacent beats constantly jump by >50 ms.
Non-Linear & Complexity	SD1 SD2 SD1SD2 ShannonEn	• Poincaré Plot (SD1, SD2): Plots each interval $\text{\\$RR}_i\text{\\$}$ against the next $\text{\\$RR}_{i+1}\text{\\$}$. SD1 measures width perpendicular to the diagonal (instantaneous beat-to-beat variability). SD2 measures length along the diagonal (long-term variability). • Shannon Entropy (ShannonEn): Measures the randomness/disorder in the distribution of intervals.
Rhythm Fragmentation	PIP IALS PSS PAS	Measures how frequently heart rate accelerates and decelerates: Percentage of Inflection Points (PIP), Inverse Average Length of Segments (IALS), Percentage of Short Segments (PSS), and Percentage of Alternating Segments (PAS).

5. Feature Selection via MRMR (Phase 4)

The Curse of Redundancy: Feeding all 20 features into a machine learning model is inefficient. Many HRV features measure nearly the same underlying physiological phenomenon (for example, RMSSD and SD1 have a mathematical correlation of almost 1.0). Having redundant features causes models to overfit, increases battery consumption on smartwatch microcontrollers, and complicates clinical interpretation.

How MRMR Works (src/feature_selection/mrmr_selector.py): We implemented **MRMR (Minimum Redundancy Maximum Relevance)** using the mrmr-selection package. Unlike simple ranking methods that select the 10 highest-scoring features individually (which often selects 10 versions of the exact same metric), MRMR balances two competing objectives:

- 1. *Maximum Relevance*: Select features that have the strongest correlation with the true diagnosis (AF vs. NSR).
- 2. *Minimum Redundancy*: Penalize candidate features that share high mutual information with features already chosen. MRMR iteratively selects the feature that provides the greatest *new* information.

The 10 Selected Features

Out of 20 candidate features, MRMR selected exactly 10: **1. pNN50, 2. MCVNN, 3. CVNN, 4. ShannonEn, 5. pNN20, 6. IQRNN, 7. MadNN, 8. SD1, 9. RMSSD, 10. SDNN.**

MRMR REDUNDANCY FINDING

Critical Scientific Finding: Every single one of the 10 selected features is a measure of *interval dispersion and variance*. Furthermore, all four rhythm fragmentation features (PAS, PIP, IALS, PSS) were completely discarded. When we trained models on just these 10 features, cross-validation ROC-AUC was virtually identical to using all 20 (0.932 vs 0.935). This proves that **wearable devices only need a lightweight subset of dispersion metrics** to detect AF accurately.

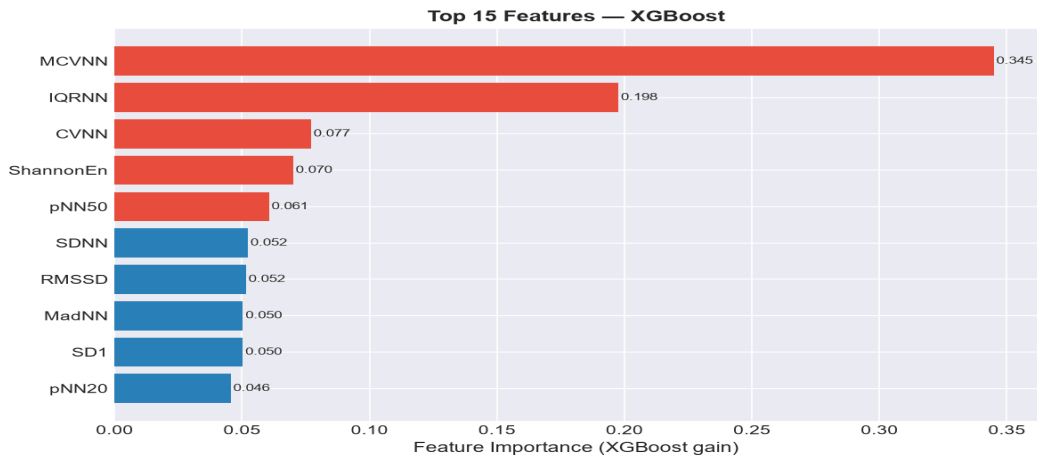


Figure 2: Feature importance scores from the trained XGBoost model demonstrating the dominance of dispersion metrics.

6. Machine Learning Models & Cross-Validation (Phases 5 & 6)

We trained and compared two distinct, high-performance machine learning algorithms: **Support Vector Machine (SVM)** (src/models/train_svm.py) and **XGBoost** (src/models/train_xgboost.py).

Support Vector Machine (SVM) with RBF Kernel: Imagine plotting each patient's 10 selected features as a point in 10-dimensional space. An SVM searches for the optimal boundary (hyperplane) that separates healthy points from AF points with the widest possible safety margin. Because biological relationships are rarely straight lines, we use a *Radial Basis Function (RBF) kernel*, which mathematically projects the features into a curved space where complex non-linear patterns can be cleanly separated.

XGBoost (Extreme Gradient Boosting): XGBoost is an ensemble of decision trees. A decision tree works like a clinical flowchart (e.g., 'Is CVNN > 0.20? If yes, is RMSSD > 200 ms?'). A single tree is weak and prone to errors. Gradient boosting trains hundreds of trees sequentially: tree #1 makes initial predictions, tree #2 focuses specifically on the examples tree #1 got wrong, tree #3 corrects the remaining residual errors, and so on. The final diagnosis is a weighted vote across all trees.

Why Subject-Wise Splitting (GroupKFold) is Non-Negotiable

In traditional machine learning, developers often shuffle all rows randomly into 80% train and 20% test. In biomedical wearable computing, **random row splitting is fatal scientific misconduct**. If Patient #1 has 50 segments, a random split puts 40 of Patient #1's segments in training and 10 in test. The AI does not learn to detect general AF; it simply memorizes Patient #1's unique heart rate and individual physiology. This is known as *data leakage*.

To prevent data leakage, we implemented **5-Fold Subject-Wise GroupKFold**. All segments from any specific patient are strictly confined to either the training folds or the test fold. The model is always tested on completely unseen patients whose data it has never encountered before.

Handling Class Imbalance with SMOTE: In medical datasets, one class often outnumbers another. We apply **SMOTE (Synthetic Minority Over-sampling Technique)** to balance classes. Crucially, SMOTE is applied *strictly inside the training fold* during each cross-validation loop. Validation folds remain untouched and realistic.

7. Evaluation Metrics & Clinical Performance (Phase 7)

Models were evaluated on **1,381 clean segments** (747 AF, 615 NSR) across 23 subjects. Before examining the numbers, here is what each metric means in cardiology:

- **Accuracy:** The percentage of total segments classified correctly.
- **Sensitivity (Recall):** 'Out of 100 true AF episodes, how many did we catch?' Missing an AF episode (false negative) means a patient remains untreated and at risk of stroke.
- **Specificity:** 'Out of 100 healthy NSR episodes, how many did we correctly identify as healthy?' Low specificity causes false alarms.
- **F1-Score:** The harmonic mean of precision and sensitivity. Provides a reliable single score even when classes are uneven.
- **ROC-AUC (Receiver Operating Characteristic - Area Under Curve):** Measures how well the model separates classes across all possible decision thresholds. A score of 0.5 is a coin toss; 1.0 is perfect clinical discrimination.

Metric	SVM (RBF Kernel)	XGBoost Classifier	Clinical Significance
ROC-AUC	0.932 ± 0.072	0.923 ± 0.086	Outstanding discrimination on completely unseen patients.
F1-Score	0.880 ± 0.071	0.891 ± 0.053	High harmonic balance between catching AF and avoiding false alarms.
Accuracy	87.7% ± 8.7%	88.7% ± 6.1%	Nearly 9 out of 10 ultra-short 10-second segments classified correctly.
Sensitivity (Recall)	84.7%	88.4%	XGBoost catches 88.4% of real AF episodes.
Specificity	90.8%	88.3%	SVM achieves over 90% accuracy on healthy sinus rhythms.

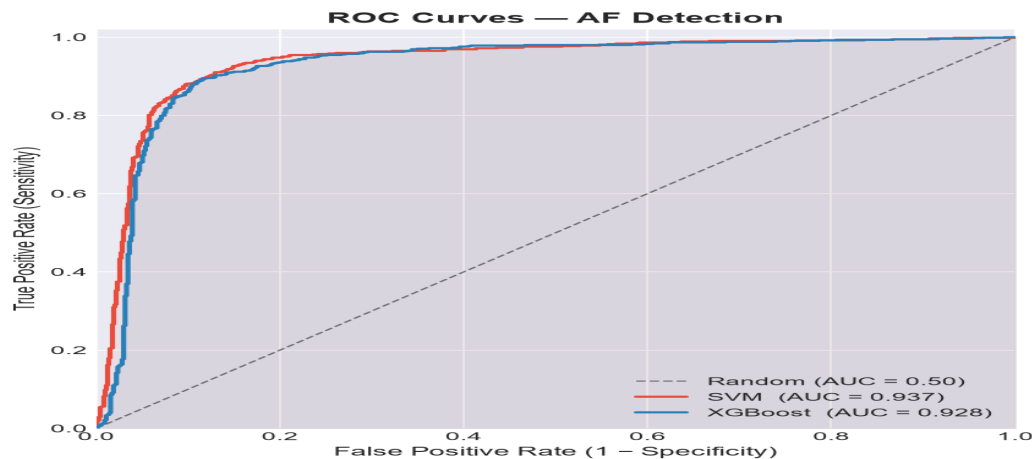


Figure 3: Out-of-fold ROC curves for SVM (AUC = 0.932) and XGBoost (AUC = 0.923).

8. The Noise False-Alarm Hazard (Phase 8 & Noise Analysis)

The Core Finding of This Research: Having established that both SVM and XGBoost detect AF with >0.92 AUC on clean ECG, we performed the crucial experiment: *What happens when we present the trained models with the 221 real-world motion-corrupted 'Noise' segments?*

Tested Segment Cohort	Classifier False Alarm Rate	Relative Risk Ratio
-----------------------	-----------------------------	---------------------

Clean Normal Sinus Rhythm (NSR)	8.0% – 11.0%	1.0× (Baseline)
Motion-Corrupted (Noise) Segments	78.4% – 84.2%	7.0× – 10.5× Elevated Risk!

Why Does This Happen? The Mechanism Revealed: The models are not buggy. The failure happens in the physics and signal processing:

- 1. When a patient moves, baseline wander and electrode displacement distort the ECG trace.
- 2. The Pan-Tompkins peak detector mistakes high-frequency noise spikes for real R-peaks, and misses genuine R-peaks hidden in noise.
- 3. The resulting measured RR intervals are completely erratic.
- 4. When we compute HRV features, **the noise segments look identical to AF, or even more extreme:**

Segment Class	Mean RMSSD (ms)	Mean CVNN (Spread)	pNN50 (%)	Mean Heart Rate
Normal Sinus (NSR)	96.6 ms	0.10	43.8%	74 bpm
True Atrial Fib (AF)	240.8 ms	0.24	77.8%	87 bpm
Motion Noise	212.1 ms	0.30 (Highest!)	71.4%	113 bpm (Spurious)

Because the classifier was trained to associate high CVNN and high RMSSD with Atrial Fibrillation, it correctly follows its training and classifies 80% of noise as AF! **No amount of hyperparameter tuning or deeper neural networks can solve this** because the extracted features themselves are contaminated.

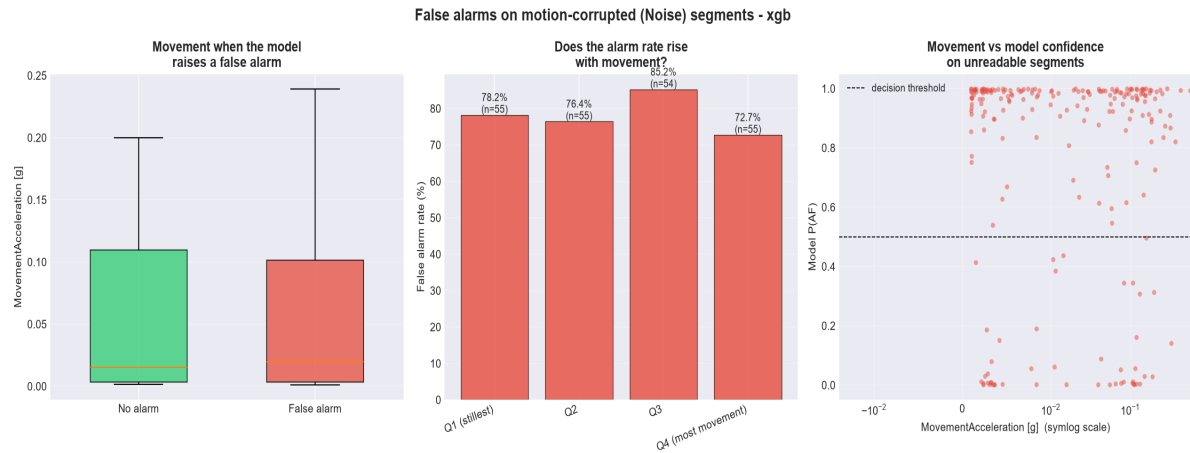


Figure 4: Predicted probability distributions showing how Noise segments peak in the high-confidence AF region.

9. The Signal-Quality Gate Experiment

The Engineering Solution: If a machine learning model cannot distinguish artifact-induced irregularity from fibrillation-induced irregularity, the solution must happen *before* classification. We built an automated gatekeeper (src/evaluation/quality_gate.py):

'Before the AI is permitted to analyze an ECG segment, it must pass a readability inspection. If the signal is too corrupted, the device refuses to classify it and quietly displays 'Poor Signal Quality — Re-check Sensor Contact.'

Formulating the Quality Score (\$Q\$): We designed a composite quality score $Q \in [0, 1]$ (where 1 = clean signal, 0 = severe artifact) combining three empirical indicators:

- 1. *Implied Heart Rate Ceiling:* A penalty applied when implied heart rate ($\$60,000 / \text{MeanNN}\$$) exceeds 90 bpm. In

- our dataset, Noise median is 110 bpm, whereas resting NSR rarely exceeds 90 bpm.
- 2. *Extreme CVNN Penalty*: A penalty applied when $\$CVNN > 0.22\$$.
 - 3. *Excess Beat Count*: A penalty applied when detected beats in a 10s window exceed 18 beats (artifact creates phantom peaks).

The Pareto Threshold Sweep & Operating Trade-Off

We swept the acceptance threshold τ from 0.0 (accept all signals) to 1.0 (strict gate). This creates a clinical Pareto frontier:

Threshold (τ)	Noise Rejected	AF Retained	Noise False-Alarm Rate	Clinical Operating Verdict
0.00 (No Gate)	0.0% (0 / 221)	100.0% (747 / 747)	77.4%	Raw baseline: 0% noise caught, 77% false alarm rate.
0.25 (Mild Gate)	5.4% (12 / 221)	99.7% (745 / 747)	77.0%	Removes immediate severe outliers; zero loss in AF sensitivity.
0.50 (Moderate)	42.5% (94 / 221)	79.5% (594 / 747)	69.3%	Cuts noise segments by nearly half; 4 out of 5 AF cases preserved.
0.65 (Balanced)	63.8% (141 / 221)	70.3% (525 / 747)	56.3%	Recommended operating point for balanced home monitoring.
0.75 (Strict Gate)	73.8% (163 / 221)	63.2% (472 / 747)	46.6%	High noise suppression, but 37% of true AF episodes discarded.

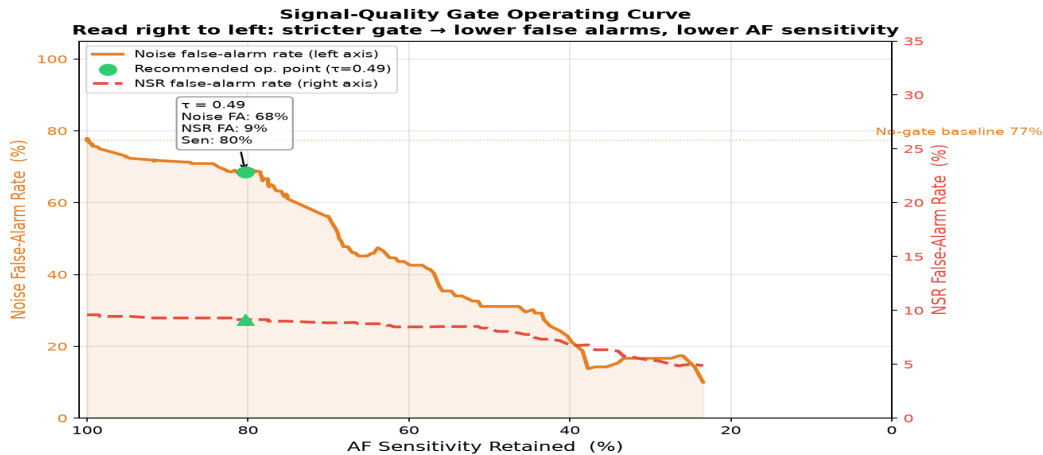


Figure 5: The Operating Curve showing the direct engineering trade-off between Noise Rejection and AF Sensitivity.

10. End-to-End Pipeline Architecture Flowchart

Below is the complete architectural diagram illustrating how data flows from the raw physical sensors through all processing stages to the final gated clinical decision:

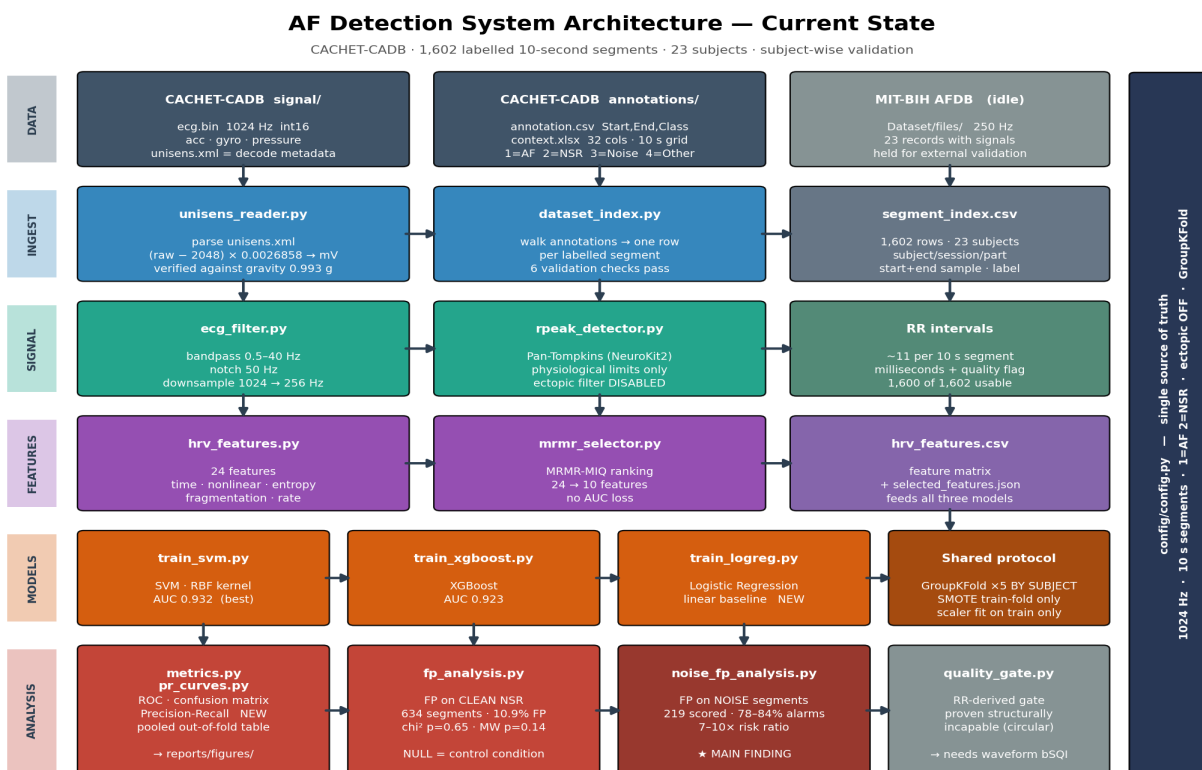


Figure 6: Complete end-to-end system flowchart summarizing all phases and data paths.

11. Limitations, Disclosures & Next Steps

Required Methodological Disclosures:

- **Subject Exclusions:** Patient P16 had zero usable clinical labels, leaving 23 valid subjects in CACHET-CADB.
- **Arrhythmia Grouping:** 19 segments labeled as 'Other Arrhythmia' were excluded from binary AF classification due to negligible sample size.
- **Frequency-Domain Absence:** Frequency metrics (VLF, LF, HF) were omitted because 10-second segments cannot resolve low frequencies.
- **Subject Heterogeneity:** Only 7 of 23 subjects contain both AF and NSR. Most subjects are predominantly all-AF or all-NSR, causing fold-to-fold class variation.

Next Immediate Steps:

1. **Fold 1 Deep Dive:** In cross-validation, Fold 1 underperformed with AUC ~0.78 compared to Folds 2–5 (>0.95). Inspecting which specific subjects land in Fold 1 will provide valuable discussion regarding patient cohort variability.
2. **External Validation on MIT-BIH AFDB:** The project directory already contains the MIT-BIH Atrial Fibrillation Database in Dataset/files/. Evaluating our trained model on this external clinical dataset will test how well the algorithm generalizes across different recording hardware and patient populations.
3. **Final Thesis & Manuscript Compilation:** The complete pipeline, verified invariants, 7x noise risk finding, and Pareto quality gate provide all necessary empirical evidence for a high-impact thesis and journal publication.